Description of the laboratory work No. 3

INTERRUPTS MANIPULATORS OF 32 MICROCONTROLLERS

1. Objectives
2. Preparation for the work
3. Workflow
   3.1. Work with a teacher
      3.1.1. STM32CubeMX
      3.1.2. Template of the code
      3.1.3. Main functions
   3.2. Work by individual task
   3.3. Literature

1. Objectives

Get to know the architecture, functionality and configuration features of the interrupts manipulators of the 32 bit microcontrollers. To develop abilities to analyze microcontrollers specifications. Get to know the software, which is needed for program of 32 bit microcontrollers.

2. Preparation for the work

We will start to work with internal and external interrupts of 32-bit microcontrollers during this laboratory work. Students will try to accomplish several tasks independently after getting to know the basics.

3. Workflow

3.1. Work with a teacher

The lecturer demonstrates boards, explains their components and students start to program individually.

3.1.1. STM32CubeMX

STM32CubeMX is a software package that allows you to perform project configuration work in the graphical interface. First, when you run the STM32CubeMX software for the first time, it will be necessary to create a new project

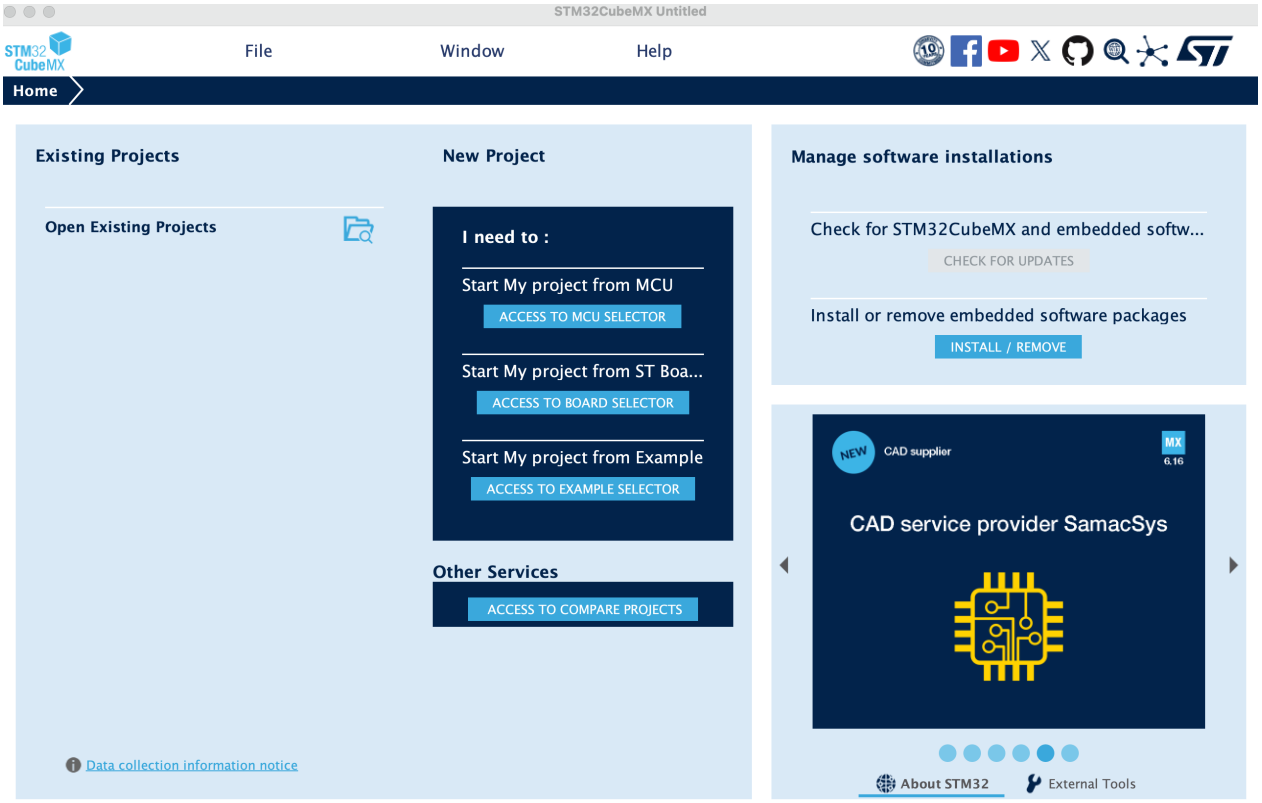


Fig. 1. The dialog window for selection of the home page

NOTE: The STM32CubeMX version may vary by laboratory, so the program window may look slightly different. Just have a look at where to choose the right configuration.

In the main configuration window, it is necessary to press the “Access to board selector” button. As a result, the board selection window will open (Fig. 2).

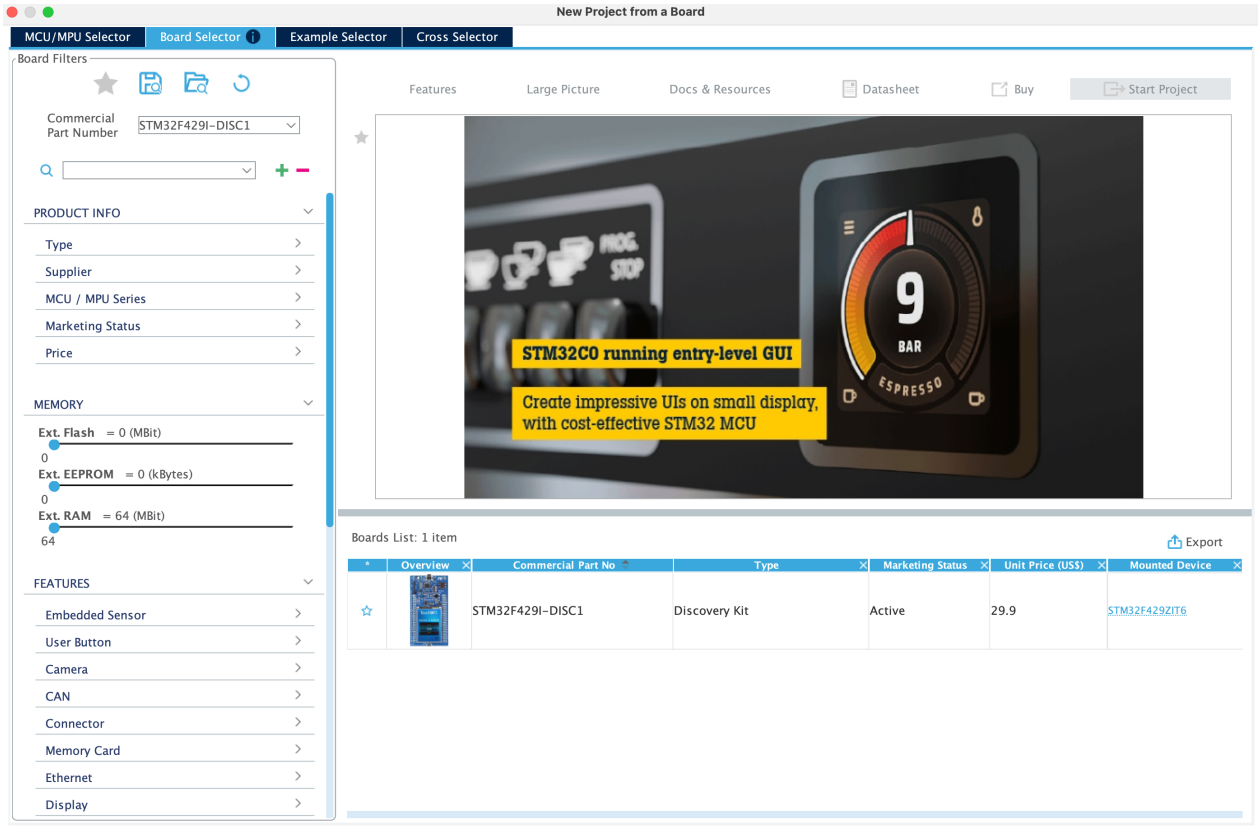


Fig. 2. Board selection window

The following steps in the board selection window should be made (Fig. 2):

1. Start to write “STM32F429I-DISC1” in the search edit box.
2. Select the necessary STM32F429I-DISC1 board from the list.
3. Press “*Start Project*” to be directed to the next configuration window (Fig. 4).

The pinout configuration window will be presented next (Fig. 3).

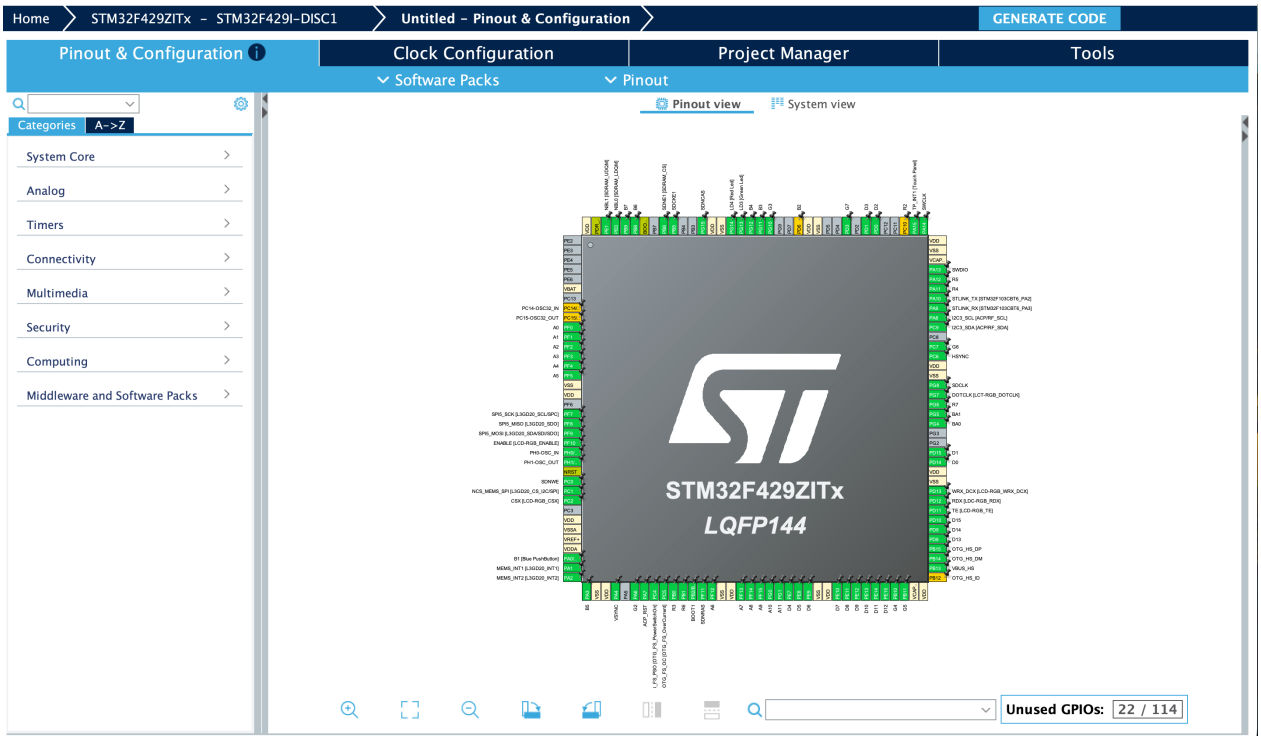


Fig. 3. Project configuration window

First, it is necessary to clear all the configuration of all pins of the microcontroller (Fig. 3): 1 – select *Pinout dropdown* menu; 2 – pick *Clear pinouts*. Then confirm the pinout clear operation (Fig. 4).

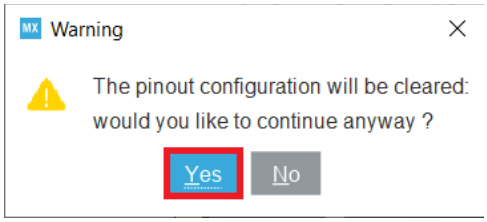


Fig. 4. Confirmation of pinouts clear

The next step is to press the left mouse button on the selected pin and to set the direction of these pins as “GPIO_Output” (Fig. 5). It is also necessary to set the label of every pin: PG13 – green LED; PG14– red LED. It could be done after pressing the right mouse button on the selected pin.

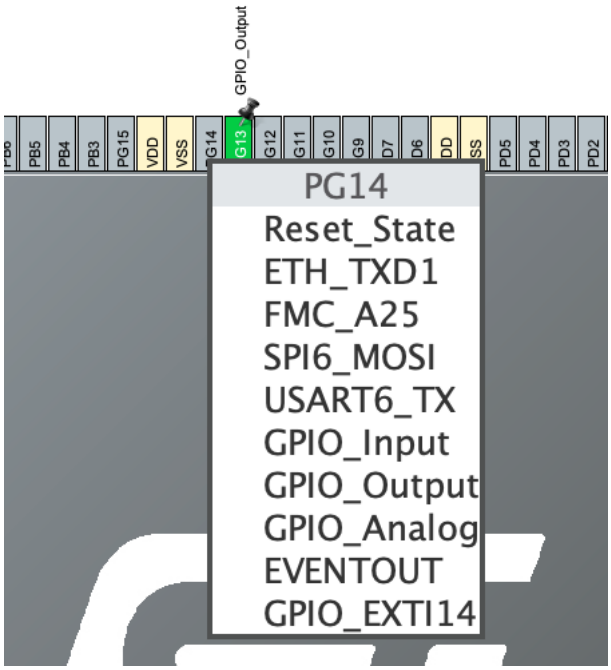


Fig. 5. Configuration of output pins

The configuration of input pin PA0 for the USER BUTTON is made using the same logic (Fig. 6).

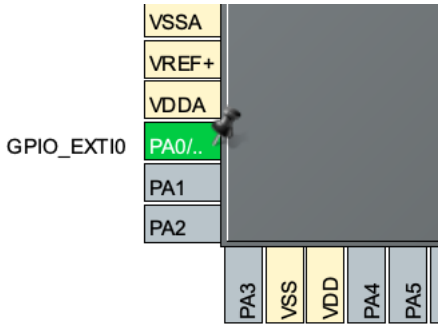


Fig. 6. Configuration of input pin

The next step is to go to the “Clock configuration” tab and to check if there are no errors (Fig. 7). There are usually no problems with the clock source, especially when performing basic operations with standard input output.

In order to use the external interrupt mechanism, additional configuration of the Nested Vectored Interrupt Controller (*NVIC*) must be performed. The NVIC is responsible for managing interrupts mechanism. Select System Core and then NVIC in the project configuration function tree (Fig. 7).

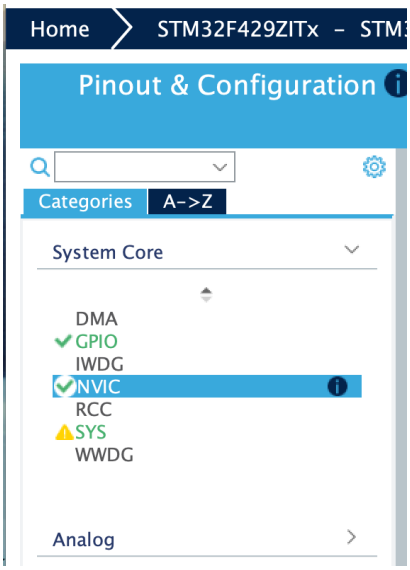


Fig. 7. *NVIC* selection window

Select the settings marked in red in the main configuration window of the *NVIC* controller (Fig. 8).

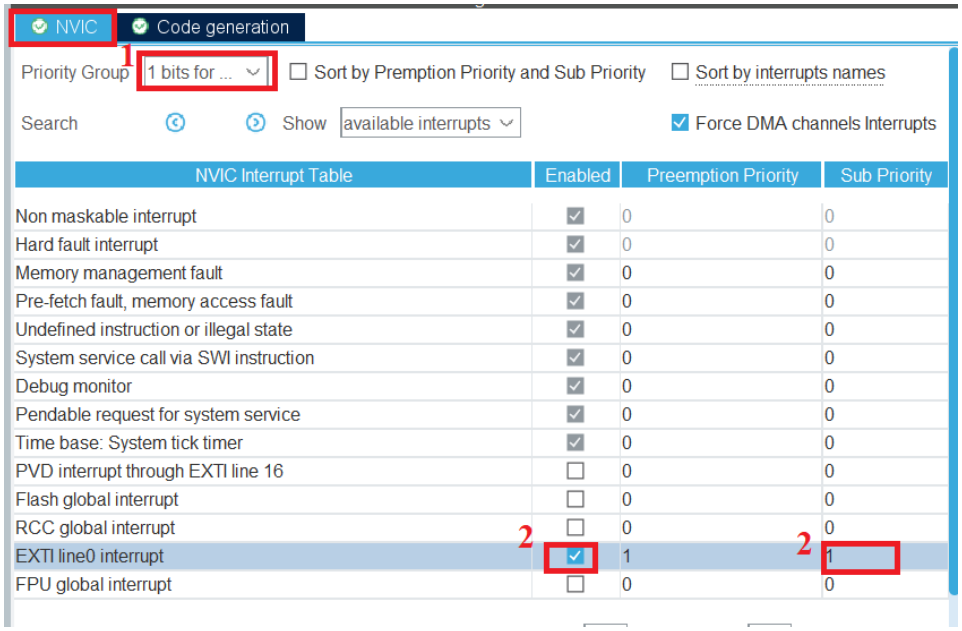


Fig. 8. Main configuration window of the *NVIC*

It is necessary to jump to the code generation section and to select the additional required configuration which is marked in red (Fig. 9).

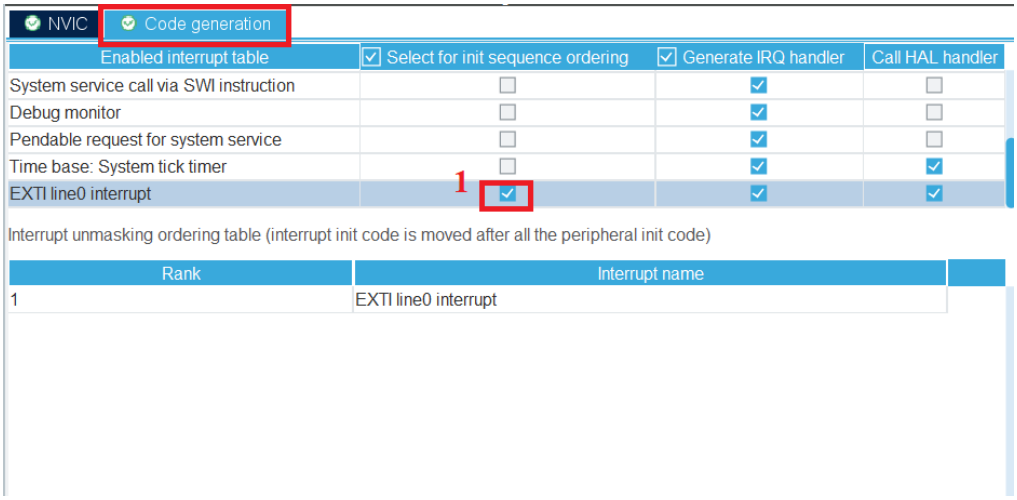


Fig. 9. Code configuration window of the *NVIC* controller

Let us move forward to the configuration of timers/counters. As an example, the configuration of the TIM2 is provided. The order of actions is presented in figure 10: 1) - select the "Timers" section; 2) select the timer/counter TIM2; 3) the source of the clock should be specified as an internal clock.

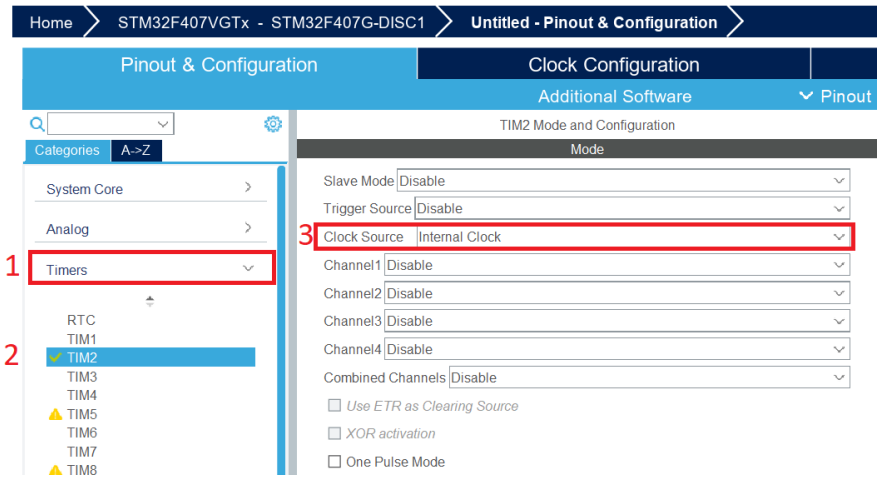


Fig. 10. Configuration window of TIM2

The configuration of internal clock source should be performed after the selection of internal clock source: 1) move to the „Clock configuration“ tab; 2) Set the frequency of the APB1 timer/counter bus equal to 72 MHz (Fig. 11).

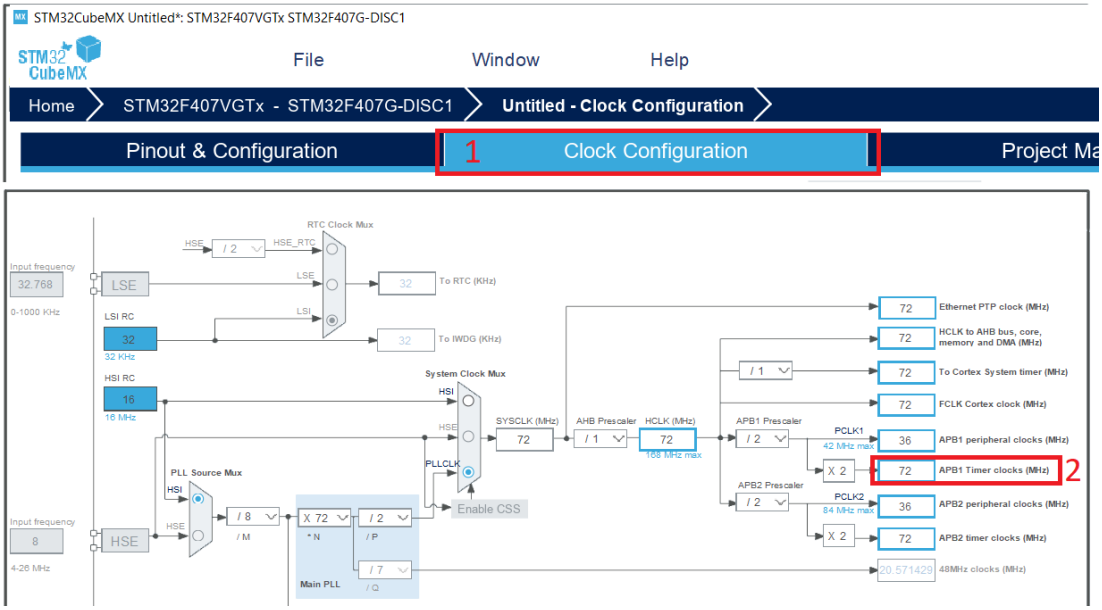


Fig. 11. Frequency configuration of APB1 timer/counter bus

When it is necessary to go back to the "Pinout and configuration" window and to go to the "Timers" configuration section (Fig. 12).

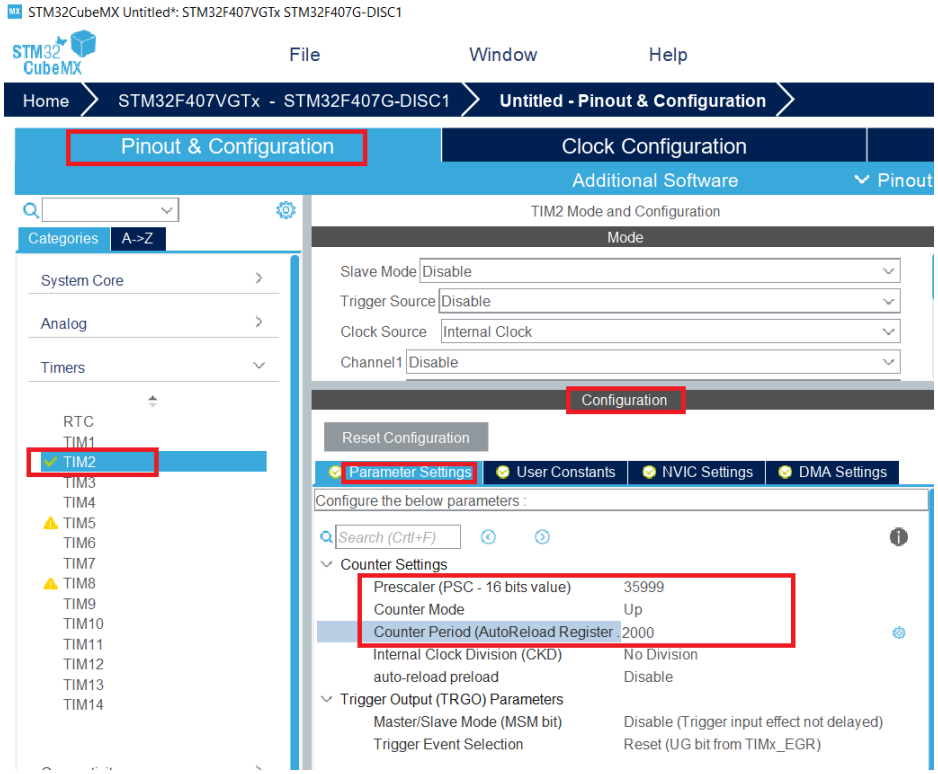


Fig. 12. TIM2 configuration section

It is necessary to select TIM2 in the Timers configuration section (Fig. 10). It is necessary to make sequential settings:

• Prescaler: 35999

• Counter mod; Up

• Counter period (autoreplied register): 2000

The 35999 value of the prescaler means that the 72 MHz system clock frequency will be divided by 36000. We will get 2000 Hz after division of 72000000/36000. As you know, 1 Hz is equal to 1 s. 2000 Hz will be equal to 0.0005 s in this case. Selecting a counter period equal to 2000 will result in one timer/counter period being exactly 1 second (0.0005 s * 2000 = 1 s). This is a timer counter value that will rise from zero to 2000 and reset within 1 second.

It is necessary to go to the section of the NVIC Settings after the configuration of frequency is finished. NVIC Settings section is responsible for the enabling of global interrupts (Fig. 13).

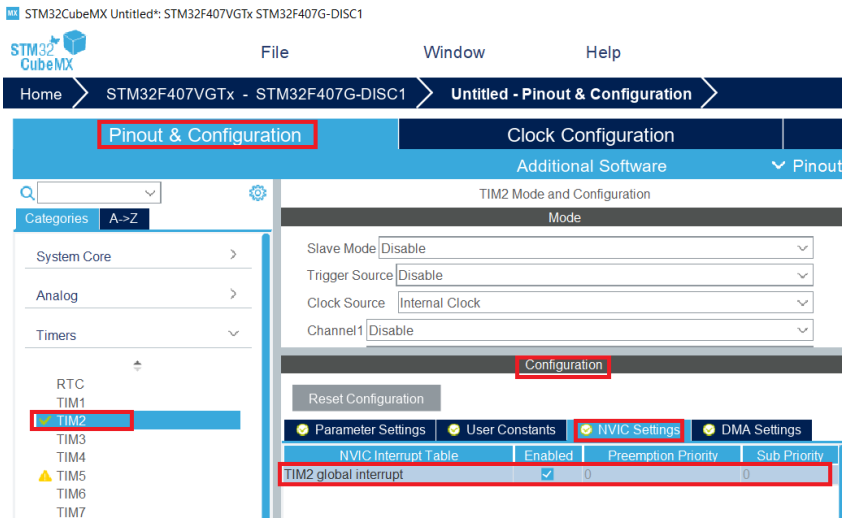


Fig. 13. Interrupts enable of the TIM2

Then it is necessary to go to the: 1) System Core; 2) NVIC; 3) NVIC; 4) Place the mark on the TIM2 Global interrupt (Fig. 14.).

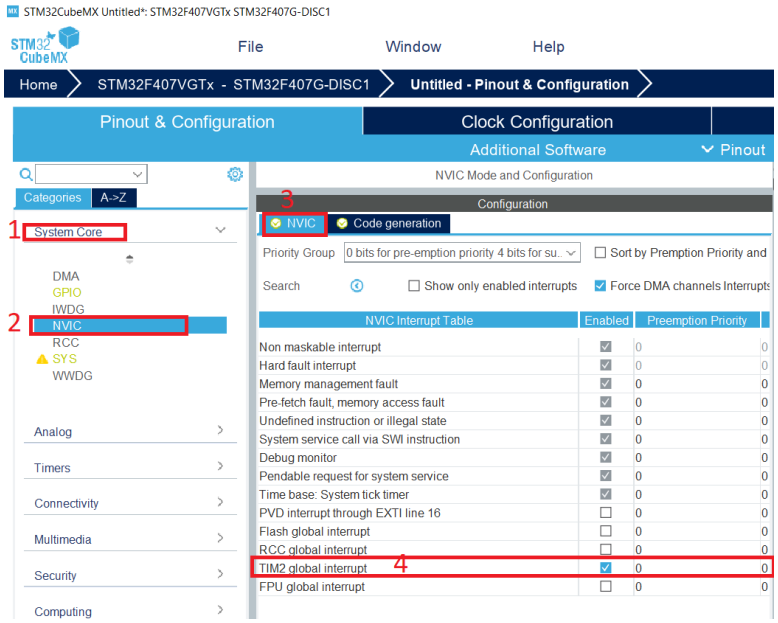


Fig. 14. Configuration window of interrupts

Lately it is necessary to go to the Code Generation tab and to place the mark on the TIM2 Global interrupt (Fig. 15).

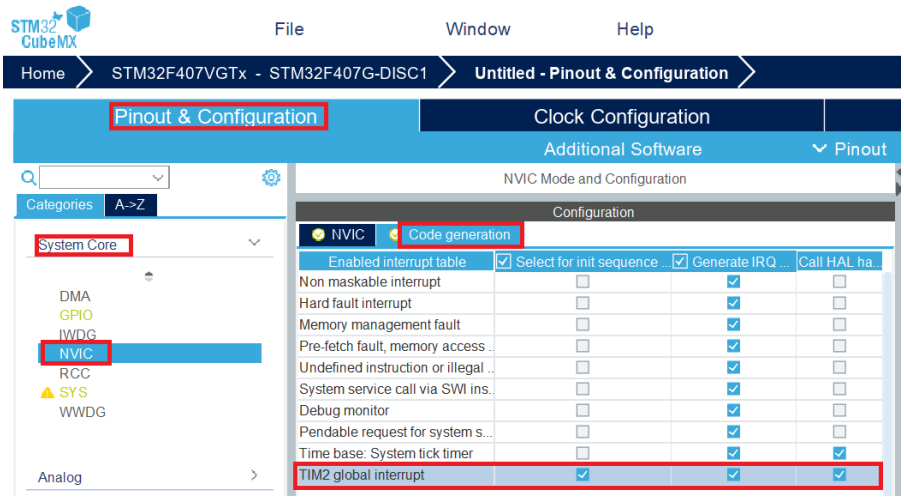


Fig. 15. Configuration window of the code generation of timer/counter interrupts

Next we will jump to the project configuration window (Fig. 16).

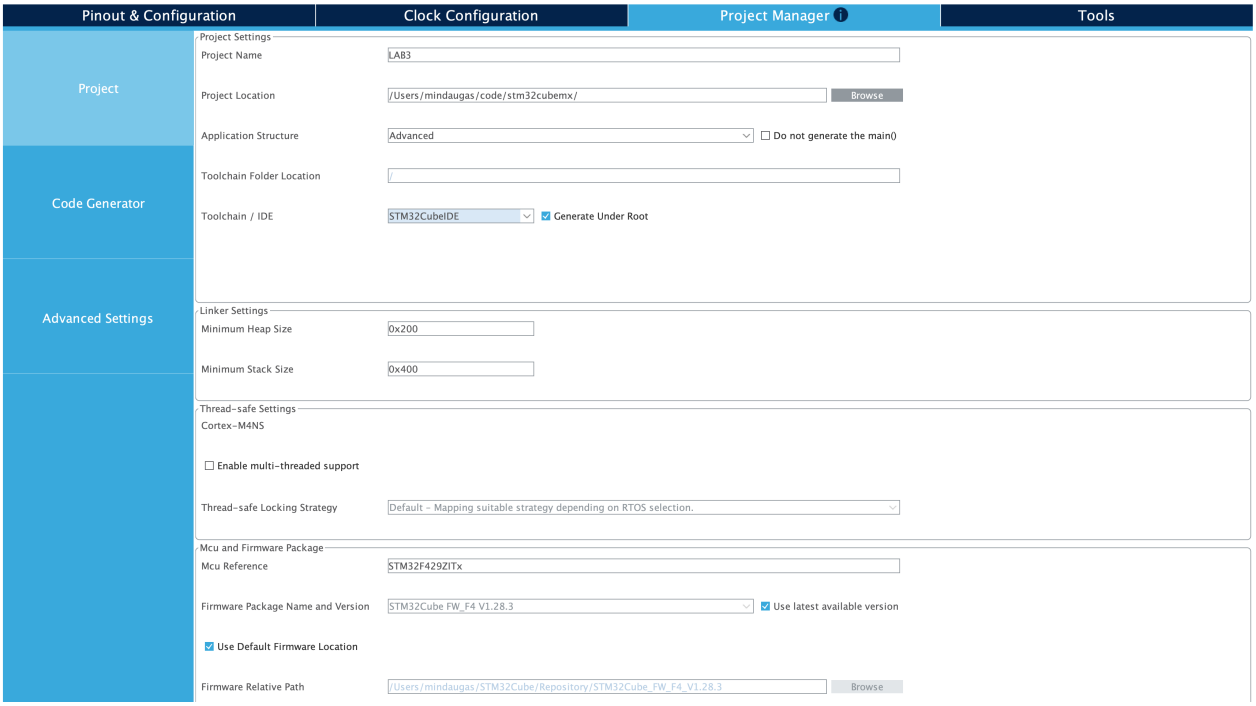


Fig. 16. Project configuration window

We will no longer need to make any changes in this window. Next we go to Menu -> Project -> Generate Code (Fig. 17).

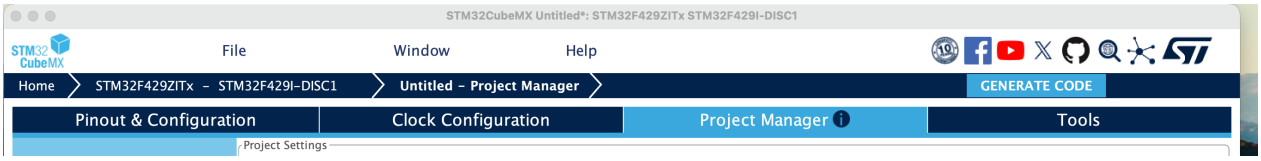


Fig. 17. Steps in order to generate the code files of the project.

For the programmer, the most important file to edit are the main.c and stm32f4xx_it.c files in third laboratory work (Fig. 18).

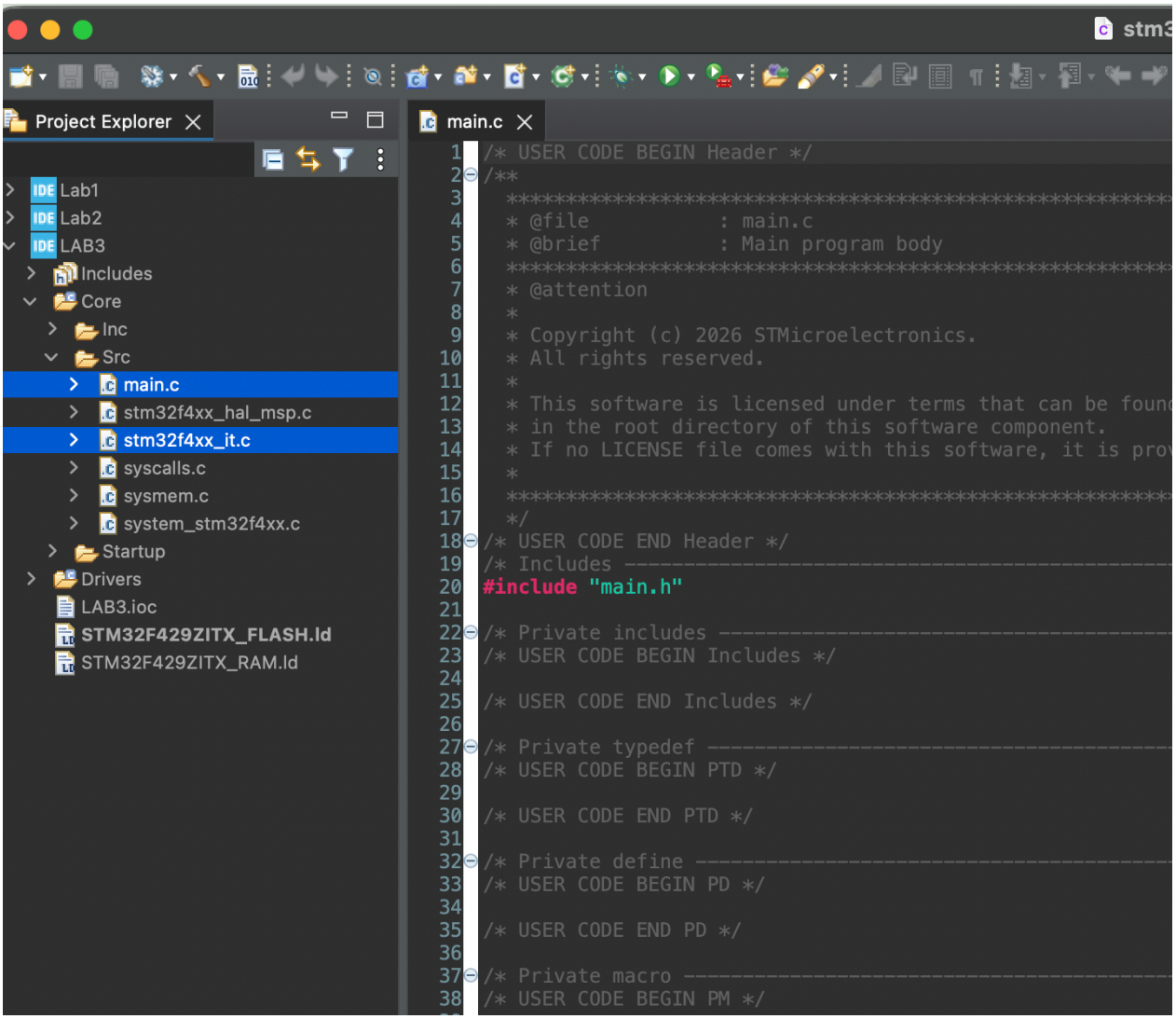


Fig. 18. The structure of the code project

The following section provides code templates in order to make your task easier by knowing there to place your written code.

3.1.2. Template of the code

I provide code templates separately for the mandatory and additional parts of the laboratory work. Students can combine everything into one project during laboratory work. It is not necessary to create two separate projects.

Code template for the mandatory part of the laboratory work:

Microcontroller systems

main.c file

```
int main(void){

while (1){
        }
}
```

stm32f4xx_it.c file

```
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "stm32f4xx_it.h"

/* STM32F4xx Peripheral Interrupt Handlers                                    */
/* Add here the Interrupt Handlers for the used peripherals.                  */
/* For the available peripheral interrupt handler names,                      */
/* please refer to the startup file (startup_stm32f4xx.s).                    */
/******************************************************************************/

/**
  * @brief This function handles EXTI line0 interrupt.
  */
void EXTI0_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI0_IRQn 0 */

  /* USER CODE END EXTI0_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(UserButton_Pin);
  /* USER CODE BEGIN EXTI0_IRQn 1 */

  /* USER CODE END EXTI0_IRQn 1 */
}

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */
```

Code template for the additional part of the laboratory work:

main.c file

```
int main(void){

  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_TIM2_Init();
  MX_NVIC_Init();
```

```
   HAL_TIM_Base_Start_IT(&htim2); // Funkcija, kuri inicializuoja TIM2

   while (1){
   }

}
```

In the *stm32fxx_it.c* file it will be necessary to configure the function TIM2_IRQHandler:

```
void TIM2_IRQHandler(void){

   HAL_TIM_IRQHandler(&htim2);
   HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_12);  // Išvado 12 įtampos keitaliojimas
   /* USER CODE END TIM2_IRQn 1 */
}
    ...
```

3.1.3. Main functions and registers

Mandatory part

- static void MX_NVIC_Init(void) – NVIC interrupt initialization function
- static void MX_GPIO_Init(void) – Pins initialization function
- HAL_NVIC_SetPriority() – Interrupt priority management function
- HAL_NVIC_EnableIRQ() – Interrupt enable function

- HAL_GPIO_WritePin() – Sets the high / low voltage level on the output
- HAL_GPIO_TogglePin() – The voltage level on the output is inverted
- HAL_GPIO_ReadPin() – Reads the level of voltage in the input

Additional part

- MX_TIM2_Init() – TIM2 timer/counter initialization function
- HAL_TIM_Base_Start_IT() – TIM2 timer/counter startup function
- void TIM2_IRQHandler() – TIM2 timer/counter interrupt enable function

3.2. Work by individual task

Students independently perform individual work assignments provided by the lecturer. Students can use any additional literature, but cannot talk to each other. When faced with difficulties and finding no solution, students can always ask specific questions to the teacher.

3.3. Literature

1. Slides of the lectures in the moodle.vgtu.lt system.
2. Code examples in the internet.
3. STM32F407. In the moodle system.
4. STM32F407 Discovery. In the moodle system.
5. UM1467 Getting started with software and firmware environments for the STM32F4DISCOVERY Kit. In the moodle system.
6. UM1472 Discovery kit for STM32F4. In the moodle system.
7. DM00031020 stm32f4x reference manual registers level. In the moodle system.